Birla Institute of Technology & Science, Pilani

Work-Integrated Learning Programmes Division
MTech in Artificial Intelligence and Machine Learning
S2_2025-2026, AIMLCZG557/AECLZG557

Assignment 2 – PS04 -Hex Game AI-MCTS [Weightage 13%]

Read through this entire document very carefully before you start!

Problem Statement

Develop an intelligent Hex-playing agent capable of selecting the best legal move within a specified time limit.

Develop an **Hex-playing AI agent using Monte Carlo Tree Search (MCTS).** The implementation should demonstrate how statistical sampling can be used to make intelligent decisions within a fixed computation time on the state space for an N×N (where $7 \leq N \leq 11$)

The objective is to maximize the probability of winning while making decisions under real-time constraints.

Problem Formulation: Write an AI agent that takes a current grid state and returns the next best move (row, col). The Agent must implement:

- Selection
- Expansion
- Simulation (Rollout)
- Backpropagation

using the Upper Confidence Bound for Trees (UCT) policy.

Valid moves (adjacencies) for a given (r,c):

the valid adjacencies for a piece at (r, c) are:

- **Orthogonal:**
  - Top (r-1, c),
  - Bottom (r+1, c),
  - Left (r, c-1),
  - Right (r, c+1),
- **Allowed Diagonals:**
  - Top-Right (r-1, c+1) and
  - Bottom-Left (r+1, c-1)
- **Forbidden Diagonals:**

- Top-Left (r-1, c-1) and
- Bottom-Right (r+1, c+1)

Sample Input:

- An integer N (where 7≤N≤11). Use this value to create the game board.
- An N×N matrix representing the board state. The initial state is all 0's.
- Current Player: 1 for Player A, 2 for Player B, 0 for empty.
- AI Agent takes the first turn to play the game. In our scenario, Player A (symbol 1) refers to an AI Agent.
- A time limit (e.g., 2000 milliseconds per move).

7 2000 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	Line 1: Board Size Line 2: Time Limit (milliseconds) Initial Board

Once the initial input is captured, the game should continue with alternate turns. The program should prompt (take input from) the user to select the next move. The program should display the current board state and collect the user input, for further executions. The program should end once the game reaches the terminal state.

Sample Output:

- A tuple (x, y) representing the chosen coordinate.
- The computed heuristic score of that move.
- Nodes Expanded
- Depth reached
- Execution Time

```
====================Turn 1==================================
Player : A (AI)
Move Selected : (3,3)
Search Algorithm : Depth-Limited Minimax + Alpha-Beta Pruning Search Depth : 6
Nodes Expanded : 24,317
Heuristic Score : 18.42
Execution Time : 1845 ms
Game Status : CONTINUE
Current Board

====================Turn 2==================================
Player: B (Human)
Move Entered: (2,4)
Move Validation: VALID
Execution Time:
```

```
Game Status: CONTINUE
Current Board
```

Prompt during the program execution

- Display current board
- <player> your move:4,1
- Entered Move: (4,1)

```
Current Board:
0 0 0 0 0 0 0
0 0 0 0 2 1 0
0 0 0 0 1 0 0
0 0 0 1 0 0 0
2 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

Player B, Enter your move: 4,1
Entered Move: (4,1)
```

Note that the input/output data shown here is only for understanding and testing, the actual file used for evaluation will be different.

Output:

Sample consolidated output. outputPS04.txt illustration.

```
====================Turn 1==================================
Player : A (AI)
Move Selected : (3,3)
Search Algorithm : Depth-Limited Minimax + Alpha-Beta Pruning Search Depth : 6
Nodes Expanded : 24,317
Heuristic Score : 18.42
Execution Time : 1845 ms
Game Status : CONTINUE
Current Board

====================Turn 2==================================
Player: B (Human)
Move Entered: (2,4)
Move Validation: VALID
Execution Time:
Game Status: CONTINUE
Current Board

……..

==================== Turn 19==============================
Player : A (AI)
```

```
Move Selected : (6,4)
Search Depth : 8
Nodes Expanded : 54,216
Heuristic Score : +INF
Execution Time : 1988 ms
Terminal State : YES
Winner : Player A
Winning Path (0,3) → (1,3) → (2,4) → (3,4) → (4,4) → (5,4) → (6,4)
Current Board ...
============================================================ GAME
OVER ============================================================
Winner : Player A
Total Turns : 19
Total AI Moves : 10
Total Human Moves : 9
Average Search Depth : 6.9
Average Nodes Expanded : 31,472
Average AI Move Time : 1,843 ms
Maximum Search Depth : 8
Maximum Nodes Expanded : 54,216
Game Result : PLAYER_A_WINS
```

Note that the input/output data shown here is only for understanding and testing, the actual file used for evaluation will be different.

Display the output in outputPS04.txt.

Example:

Below example is for the 5X5 grid representation of the game.
Player A (1) needs to connect Row 0 to Row 4.
Player B (2) needs to connect Column 0 to Column 4.

Initial State:

```
[
  [0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0],
]
```

Mid- game state (After 8 Turns)

- **Player A** hit that wall and had to route around it to the right: (0,2), (1,2), (1,3), (2,3)
- **Player B** tried to build a horizontal wall across the middle: (2,0), (2,1), (2,2), and dropped down to (3,2).

Player A: Turn 9. Moves to (3,3)

```
[
  [0, 0, 1, 0, 0],
  [0, 0, 1, 1, 0],
  [2, 2, 2, 1, 0],
  [0, 0, 2, 1, 0],
  [0, 0, 0, 0, 0]
]
```

Next step Player B should try to block Player A reaching the terminal state.

It is Turn 10. Player B is in trouble. Player A is at (3,3) and is only one row away from winning. Because of the valid adjacency rules, Player A has **two** valid winning moves to reach the bottom row:

1. **Orthogonal Down:** (4, 3)
2. **Bottom-Left Diagonal:** (4, 2) (using the r+1,c−1 rule).

Player B must try to block. Let's say B places a piece at (4,3) to block the straight path.

Player B: Move (4,3)

```
[
  [0, 0, 1, 0, 0],
  [0, 0, 1, 1, 0],
  [2, 2, 2, 1, 0],
  [0, 0, 2, 1, 0],
  [0, 0, 0, 2, 0]
]
```

Terminal State:
Player A simply uses the valid **Bottom-Left diagonal** adjacency to bypass the block, placing their piece at (4,2).
Player A Move: (4,2)

```
[
  [0, 0, 1, 0, 0],
  [0, 0, 1, 1, 0],
  [2, 2, 2, 1, 0],
  [0, 0, 2, 1, 0],
  [0, 0, 1, 2, 0]
]
```

The Winning State

```
[
  [0, 0, 1, 0, 0],
  [0, 0, 1, 1, 0],
  [2, 2, 2, 1, 0],
  [0, 0, 2, 1, 0],
  [0, 0, 1, 2, 0]
]
```

Tracing Player A's Valid Winning Path:

- Start: (0, 2)
- Orthogonal Down to (1, 2)
- Orthogonal Right to (1, 3)
- Orthogonal Down to (2, 3)
- Orthogonal Down to (3, 3)
- **Bottom-Left Diagonal** to (4, 2) — *Match over.*

Evaluating the Moves from (3, 3)

If Player A's last piece is at **(3, 3)**, let's look at the cells in the bottom row (Row 4):

1. **(4, 3) (Orthogonal Bottom):** Valid. It connects directly below.
2. **(4, 2) (Bottom-Left Diagonal):** Valid. It follows the allowed (r+1, c-1) diagonal rule.
3. **(4, 4) (Bottom-Right Diagonal): INVALID.** It falls under the forbidden (r+1, c+1) diagonal rule.

Background

Hex is a two-player, deterministic, zero-sum board game invented by Piet Hein and independently rediscovered by John Nash. It is widely studied in Artificial Intelligence because of its enormous search space, strategic complexity, and the **absence of draw states**.

Unlike games such as Chess or Tic-Tac-Toe, every completed game of Hex has exactly one winner. This property makes Hex an excellent benchmark for adversarial search algorithms and heuristic evaluation functions.

While the game conceptually takes place on a hexagonal grid (to prevent theoretical draw states), you will represent the board programmatically as a standard 2D array (matrix).

To simulate hexagonal connectivity on a square grid, a piece at index (r,c) is connected to exactly **six neighbors**.

Valid Adjacencies for (r,c):

- **Orthogonal:** (r−1,c) [Top], (r+1,c) [Bottom], (r,c−1) [Left], (r,c+1) [Right]
- **Diagonal:** (r−1,c+1) [Top-Right], (r+1,c−1) [Bottom-Left]
- *Constraint:* Top-Left and Bottom-Right diagonals are **invalid** and do not connect.

Deliverables:

- PDF document **designPS04_<group id>.pdf** detailing your solution.
- **[Group id] _Contribution.xlsx** mentioning the contribution of each student in terms of percentage of work done. Columns must be “Student Registration Number”, “Name”, “Percentage of contribution out of 100%”. If a student did not contribute at all, it will be 0%, if all contributed then 100% for all.
- **inputPS04.txt** file used for testing
- **outputPS04.txt** file generated while testing
- .py file containing the python code. Do not fragment your code into multiple files
- Zip all of the above files including the design document in a folder with the name:
    - **[Group id] _A2_PS04.zip** and submit the zipped file.
    - Group Id should be given as **Gxxx** where **xxx** is your group number. For example, if

your group is 26, then you will enter **G026** as your group id.

- **NOTE:** The program must implement at least 2 test cases and handle edge cases such as invalid moves entered by the user, no valid input and so on, such that the program should end smoothly

Instructions:

- It is compulsory to make use of the data structure(s) / algorithms mentioned in the problem statement.
- Ensure that all data structure insert and delete operations throw appropriate messages when their capacity is empty or full. Also ensure basic error handling is implemented.
- For the purposes of testing, you may implement some functions to print the data structures or other test data. But all such functions must be commented before submission.
- Make sure that your read, understand, and follow all the instructions
- Ensure that the input, prompt and output file guidelines are adhered to. Deviations from the mentioned formats will not be entertained.
- The input, prompt and output samples shown here are only a representation of the syntax to be used. Actual files used to evaluate the submissions will be different. Hence, do not hard code any values into the code.
- Please note that the design document must include:
    - One alternate way of modeling the problem with the performance implications.
    - Writing a good technical report and well documented code is an art. Your report cannot exceed 4 pages. Your code must be modular and quite well documented.
    - You may ask queries in this dedicated discussion section. Beware that only hints will be provided and queries asked **in other channels like MS Teams, emails, Taxila inbox will not be responded to**.

Deadline

- The strict deadline for submission of the assignment is **17th Aug 2026 11:55 PM IST (Monday).**
- The deadline has been set considering extra days from the regular duration in order to accommodate any challenges you might face. No further extensions will be entertained.
- Late submissions will be evaluated as below. Submissions made between:
    - 17th Aug 2026 11:56PM to 18th Aug 2026 11:55PM → Deduct 2M for late submission and evaluation for 10 marks.
    - 18th Aug 2026 11:56PM to 19th June 2026 11:55PM → Deduct 6M for late submission and evaluation for 6marks.
    - Beyond 19th Aug 2026 11:55PM → No evaluations, 0 marks will be awarded.

How to submit

- This is a group assignment.
- Each group has to make one submission (only one, no resubmission) of solutions.
- Each group should zip all the deliverables in one zip file and name the zipped file as [Group id] _A2_PS04_XXXXXXXXXX.zip and submit the zipped file.
- Group Id should be given as Gxxx where xxx is your group number. For example, if your group is 26, then you will enter G026 as your group id.
- Assignments should be submitted via Taxila > Assignment section. Assignments submitted via other means like email etc. will not be graded.

Evaluation

- The assignment carries **13 Marks.**
- Grading will depend on:
- Fully executable code with all functionality working as expected
    - Well-structured and commented code
    - Accuracy of the design document.
    - Every bug in the functionality will have negative marking.
    - Marks will be deducted if your program fails to read the input file used for evaluation due to change / deviation from the required syntax.
    - Provide the time complexity analysis of the problem.
    - What if the board size is more than 11X11 grid. Would this program complete its execution in a deterministic amount of time? What suitable algorithms could be used in the large space games and why.
    - Explain any paper or article of your that is using Game theory in the present AI trends such as LLMs, Reinforcement Learning, Fake news generation and detection.
    - Provide the trade-offs mentioned above in this document.
- We encourage students to take the upcoming assignments and examinations seriously and submit only original work. Please note that plagiarism in assignments will be taken seriously. Refer to the detailed policy here.
- Source code files which contain compilation errors will get at most 25% of the value of that question.

Readings

Artificial Intelligence: A Modern Approach Textbook by Peter Norvig and Stuart J. Russell, 4th Edition.